

浙江大学



实训报告

课程名称： 计算机控制系统设计与实践

项目名称： PLC 温度控制系统

组号： 2

组长： 李丰克 3230105182

组员：

2026 年 5 月

一、概述

本温度控制系统基于西门子 S7-1500PLC 进行设计和开发，不仅实现了对温度系统的控制同时也配备了监控系统。系统支持手自动切换，手动模式下用户可自行输入 0-100%的功率，系统可以将其转化为可直接输出给温度对象的数字量，温度对象对应改变电流大小以改变加热功率。在自动模式下，使用增量式 PID 控制，实时采集温度数据形成动态调节闭环，使温度对象能够自行调整加热功率使温度达到设定值，同时配有无扰动切换防止控制器输出发生跳变。系统还有声光报警系统，当温度超上限或者低下限时，对应报警灯会闪烁并蜂鸣器产生不同频率的报警，并在 wincc 界面同步添加报警日志，同时还配有报警消音和保持按键。同时当传感器掉线数据采集异常时会触发安全联锁切断加热并报警。在 wincc 界面中绘制出了当前温度值、设定值、控制器输出的曲线并同步配有历史曲线查询功能。

二、系统功能介绍

(1) 温度自动控制：用增量式 PID，系统将计算监测的温度值与设定值之间的误差代入进 PID 计算公式进行运算，计算得到当前的控制器输出，调节电流大小以调节加热功率，使温度稳定在设定值。

(2) WinCC 界面：系统设置了 WinCC 界面对系统的运行状态、控制效果等进行监控，还可以进行系统参数的调整。在 WinnCC 中配备了启动界面、参数调整界面、曲线界面、报警信息界面四个界面。

(3) 系统整体启动与停止：不仅在机柜上有物理启停按钮，在上位机上也有启停按钮，双方互不影响，都可控制系统整体的启动与停止，同时在上位机有表示系统运行状态的指示灯，当启动状态时灯为绿色，当停止状态时灯为红色。

(4) 模拟量采集滤波算法：由温度对象直接读取到的温度值是数字量，我们设计算法将其转换为对应量程下的模拟量并设计滤波算法来减少噪声影响，使采集的数据变化更平滑。

(5) 温度手动控制：从用户角度来看，输入一个单位为百分比(%)的量来表示手动控制的控制器输出功率，手动控制算法能够将其换算为温度对象可直接读取的数字量。

(6) 手自动无扰动切换：当系统为自动模式时，手动控制器输出始终追踪自动控制器输出；当系统为手动模式时，自动控制器的设定值始终追踪温度监测值，并且切到自动的一瞬间将此时刻的手动输出值赋值给自动控制输出。这样手动自动互相切换时最终的控制器输出都不

会跳变。

(7) 算法内部保护：因为理论控制器的输出单位是百分比(%), 输出值的范围为 0-100, 因此无论是在手动控制还是自动控制都进行了 0-100 的上下限限幅。

(8) 参数在线调整：在上位机上可实时调整手动控制器输出、自动控制器的设定值、PID 参数、温度报警上下限、温度量程上下限、报警回差以及手自动切换, 并且在手自动切换时添加了一个确认窗口防止误触。

(9) 数据曲线与历史记录查询导出：在上位机上不仅实时展示控制器输出值和温度监测值, 也控制器输出、温度值、设定值绘制为曲线, 同时设计了历史记录查询选项、按下后可将曲线从实时监控模式更改为历史记录曲线。同时对这些数据有时刻记录并可导出为 txt 文件。

(10) 上下限报警：当温度超上限时, 对应上限的报警灯闪烁, 蜂鸣器以 3HZ 报警; 当温度低于下限时, 对应下限的报警灯闪烁、蜂鸣器以 1HZ 报警; 同时上位机界面的报警指示灯也同步闪烁, 并且在报警信息界面同步添加报警记录。

(11) 报警消音和保持：在上位机上有报警消音和保持按钮。当报警时按下消音键后, 蜂鸣器会静音但是报警灯仍然保持闪烁, 再按下后蜂鸣器重新开始报警。在报警时按下保持键后, 系统将一直保持当前的声光报警状态, 即使此时的温度已经到达正常范围, 直到再次按下保持键后停止。

(12) 传感器掉线安全连锁：当测量的温度超过量程或者出现非常大的跳变时, 都说明是温度传感器出现了问题, 此时系统会强制切换为手动模式并将输出设为 0%以切断加热, 同时声光报警。并在上位机添加报警记录。

(13) 参数掉电保持：在地址分配时, 将 MB0 到 MB46 的地址设置为保持性存储器, 并且将 Kp、Ti、Td、回差、设定值、手动输出值、温度量程上下限、温度报警上下限这些变量定义在此地址范围内。即使 PLC 的 cpu 断电重启, 这些变量的值也不会重置, 实现了掉电保持。

三、主要功能的实现方案

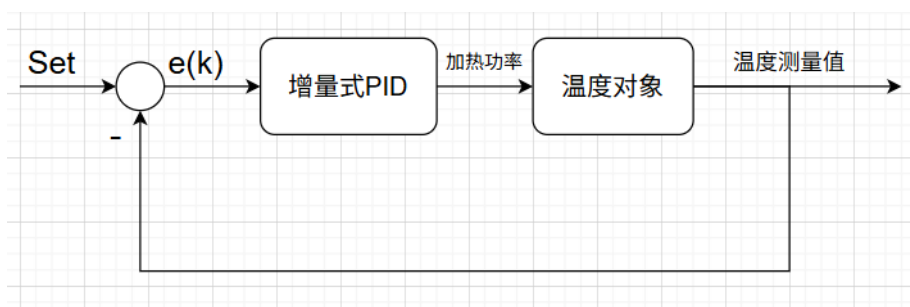
PLC 架构如下：



3.1 温度自动控制

在本次实验中，我们将测得温度作为控制输入，加热功率(1-100%)作为控制输出，用增量式 PID 计算控制输出。

控制系统基本框图如下：



由于实际控制系统为离散系统，因此，我们使用离散增量 PID 公式进行计算：

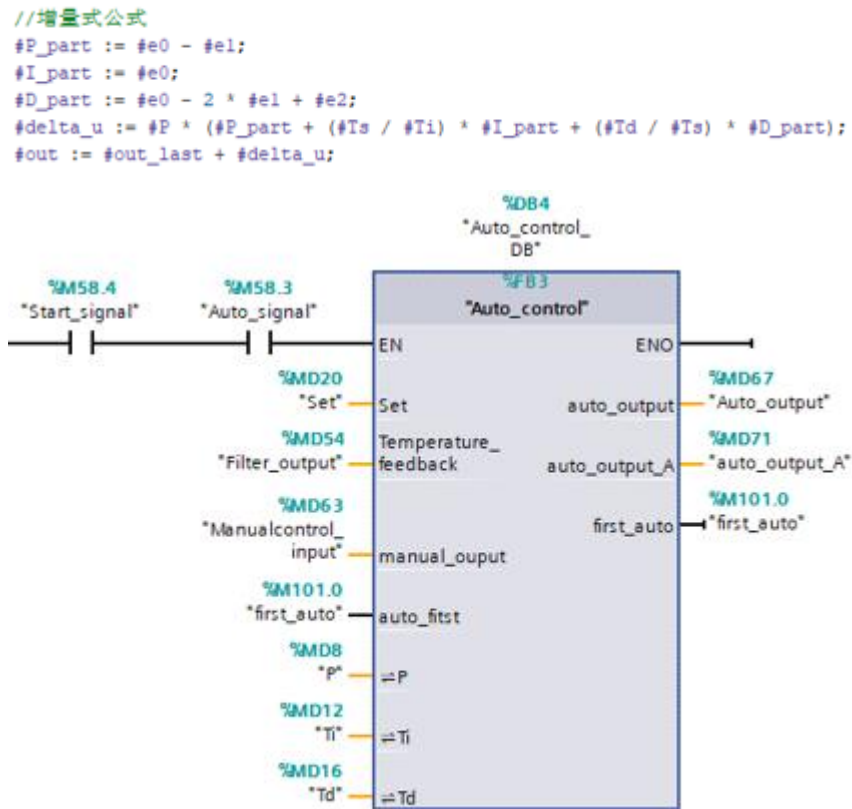
$$u(k) = K_p \left(e(k) + \frac{T}{T_i} \sum_{n=0}^k e(n) + \frac{T_d}{T} (e(k) - e(k-1)) \right)$$

比例系数 K_p ，积分时间 T_i ，微分时间 T_d ，采样时间 T 。

$$u(k) = u(k-1) + \Delta u(k)$$

$$\Delta u(k) = K_p[(e(k) - e(k-1)) + \frac{T}{T_i} e(k) + \frac{T_d}{T}(e(k) - 2e(k-1) + e(k-2))]$$

编写成 SCL 代码如图所示：



调试得到的 PID 参数依次为 $K_p = 10.0$, $T_i = 60.0s$, $T_d = 2.0s$



控制效果如图所示，为设定值从 45℃上升到 50℃的曲线。

3.2 温度手动控制

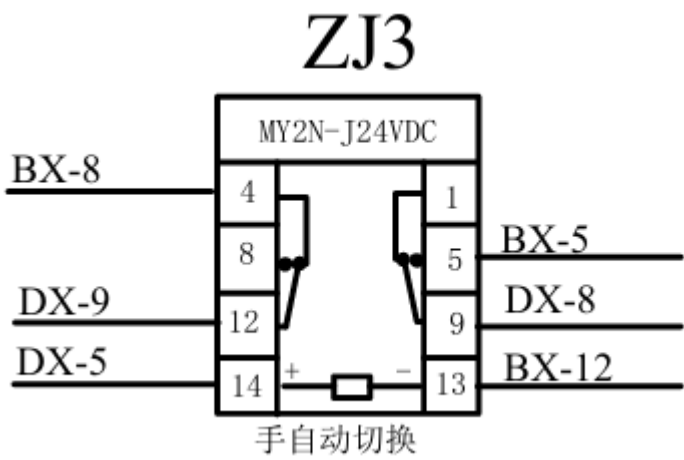
手动输入 0-100%的功率，算法将其映射到范围为 0-27648 的数字量

```
1 #Temp_input := #Manual_input;
2 IF #Temp_input>=100.0 THEN
3     #Temp_input := 100;
4 END_IF;
5 IF #Temp_input<0.001 THEN
6     #Temp_input := 0.001;
7 END_IF;
8 #Manual_output := REAL_TO_INT(#Temp_input / 100.0 * 27648.0);
9
```

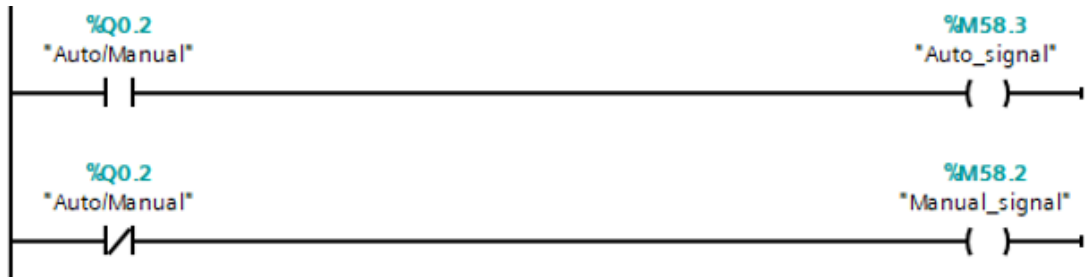
同时手动和自动的输出都做了 0-100%的限幅。

3.3 手/自动无扰动切换

定义变量 Auto/Manual 表示手自动控制，同时此变量也是外部变量，与 DO 相关联并接到继电器 ZJ3-14 上,ZJ3-12 与 ZJ3-9 分别接手动指示灯与自动指示灯,当 Auto/Manual=TRUE 时，表示为自动，当 Auto/Manual=FALSE 时，表示为手动，同时控制继电器使对应灯亮。



在代码中为方便区分定义两个内部变量 Auto_signal 和 Manual_signal 分别表示状态。

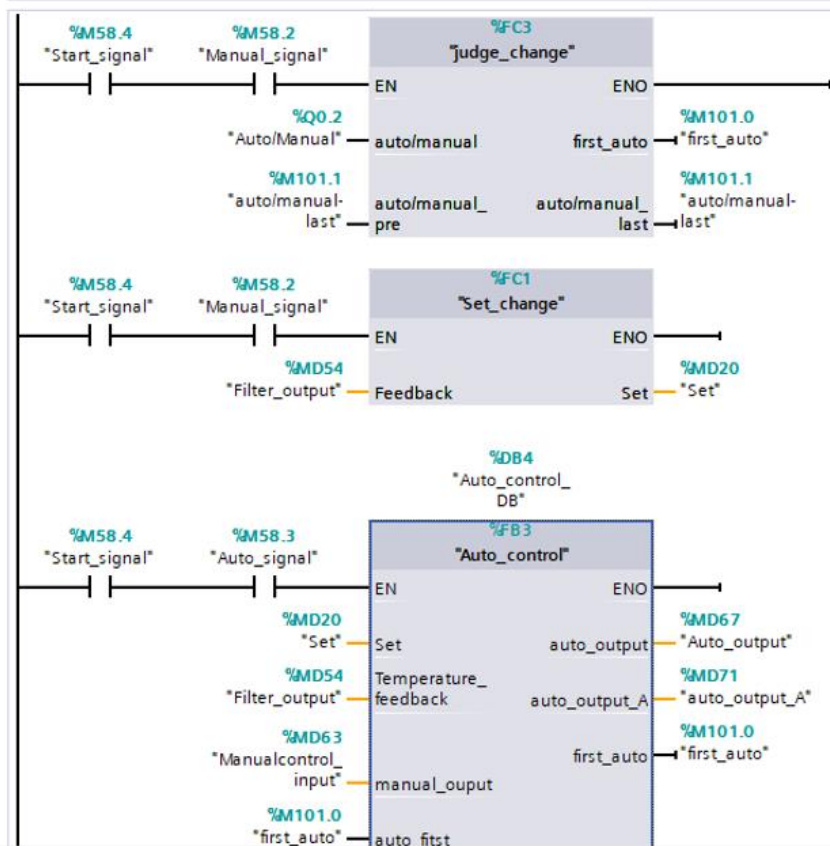


实现无扰动切换要满足三个要求：手动模式时设定值追踪测量值、自动模式时手动输出追踪自动控制输出、首次切换到自动时要以之前最后一刻的手动控制值为基础加增量。

在我们的程序中是这样实现的，其中 Set_change 函数负责使设定值追踪测量值。

▼ 程序段 4：自动控制

注释

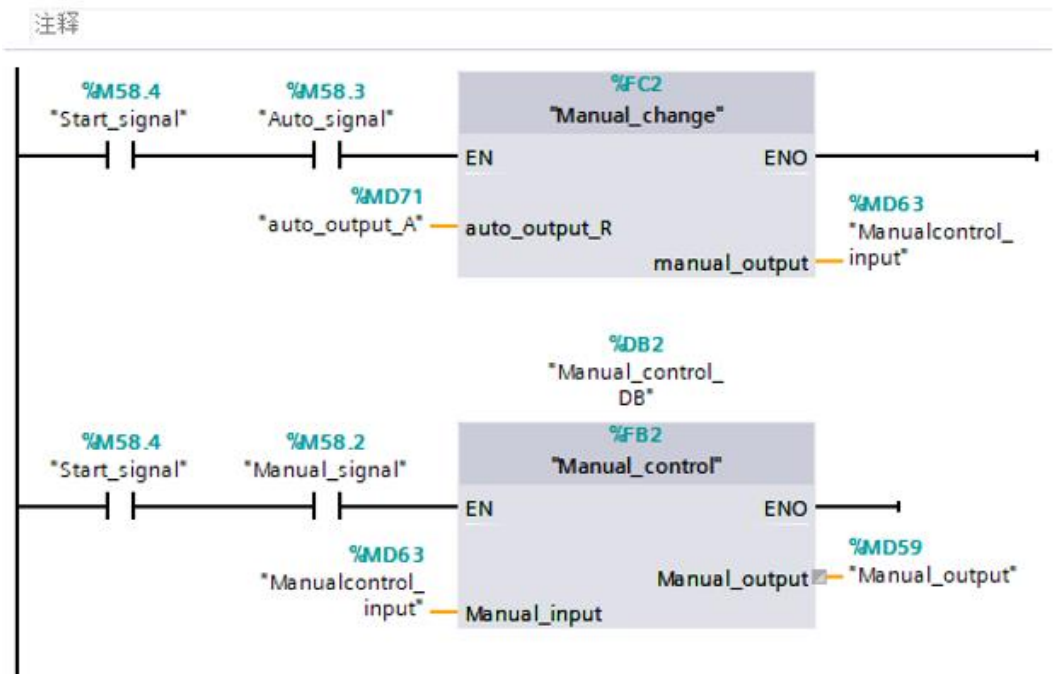


Judge_change 函数负责标志是否为第一次切换自动模式。其与变量 first_auto 有关，在手动模式下，fitst_auto:=FALSE，然后在自动控制代码里有如下处理：

```
//求误差
#FLAG := #auto_fitst;
#Ts := 0.1;
#e0 := #Set - #Temperature_feedback;
//如果本次是自动且上次是手动，说明首次切换为自动
IF NOT #FLAG THEN
    #e2 := #e1;
    #e1 := #e0;
    #out := #manual_ouput;
    #first_auto := TRUE;
ELSE
    //增量式公式
    #P_part := #e0 - #e1;
    #I_part := #e0;
    #D_part := #e0 - 2 * #e1 + #e2;
    #delta_u := #P * (#P_part + (#Ts / #Ti) * #I_part + (#Td / #Ts) * #D_part);
    #out := #out_last + #delta_u;
END_IF;
#out_max := 100.0;
#out_min := 0.0;
//数据更新
#e2 := #e1;
#e1 := #e0;
#out_last := #out;
//转数字量
#auto_output_A := #out;
#auto_output := REAL_TO_INT(#out / 100.0 * 27648.0);
```

首次切换为自动时，first_auto 为 FALSE，代码首次循环执行将手动控制值赋值给自动控制值的操作，随后将 first_auto 置为 TRUE，进行正常的增量式 PID 运算。

程序段 5： 手动控制



手动控制如上图所示，Manual_change 函数负责在自动模式时将手动控制值追踪自动控制输出值。

为了保证自动控制和手动控制能独立运行，也为了防止输出给对象的变量地址冲突，最终给控制器的输出要先经过一个筛选函数。

```
1 IF # "auto/manual" THEN
2   #control_output := #auto_out;
3   #control_output_A := #auto_out_A;
4 END_IF;
5 IF NOT # "auto/manual" THEN
6   #control_output := #manual_out;
7   #control_output_A := #manual_out_A;
8 END_IF;
```

这样就实现了当手自动切换时控制器最终输出不发生跳变，即无扰动切换。

3.4 滤波算法

一阶低通滤波算法 $output = Temp_in * \alpha + (1 - \alpha) * output_pre$ ，其中 α 为滤波系数，这里取 0.3。

直接读取到的值是一个范围为 0-27648 的数字量，先将其映射到 0-1，再将其映射到给定的温度量程内(Temp_low, Temp_high)。最后映射得到的模拟量再进行一阶低通滤波。

```

#Temp_ana:=NORM_X(MIN:=0,VALUE:=#Temp_in,MAX:=27648);
#Ana_In := SCALE_X(MIN := #temp_min, VALUE := #Temp_ana, MAX := #temp_max);
// 如果是第一个数据, pre就是本次的数据
IF NOT #first_flag THEN
    #Output_pre := #Ana_In;
    #first_flag := TRUE;
END_IF;
//计算输出
#Output := #Temp_a * #Ana_In + (1 - #Temp_a) * #Output_pre;
#Output_pre := #Output;

```

这样就将读取的数字量转化为模拟量, 同时滤波可以大幅降低噪声影响, 使测量平滑。

3.5 历史数据查询

在 PC_station 的“数据记录”中, 可以指定存储的地址以及存储的格式, 在“记录变量中”, 用户可以自行设定需要存储的变量, 这里我们选择存储温度测量值与控制器输出值, 格式选择 txt, 可以得到:

```

["VarName" "TimeString" "VarValue" "Validity" "Time_ms"
"Filter_output" "2026-05-19 21:28:59" 30.34064 1 46161895130.3009
"control_output_A" "2026-05-19 21:28:59" 110.6739 1 46161895130.3009
"Filter_output" "2026-05-19 21:29:00" 30.38543 1 46161895141.956
"control_output_A" "2026-05-19 21:29:00" 111.7676 1 46161895141.956
"Filter_output" "2026-05-19 21:29:01" 30.43394 1 46161895153.5995
"control_output_A" "2026-05-19 21:29:01" 112.93 1 46161895153.5995
"Filter_output" "2026-05-19 21:29:02" 30.48285 1 46161895165.2083
"control_output_A" "2026-05-19 21:29:02" 103.6429 1 46161895165.2083
"Filter_output" "2026-05-19 21:29:03" 30.53312 1 46161895176.8403
"control_output_A" "2026-05-19 21:29:03" 102.9246 1 46161895176.8403
"Filter_output" "2026-05-19 21:29:04" 30.58834 1 46161895188.4606
"control_output_A" "2026-05-19 21:29:04" 103.5061 1 46161895188.4606
"Filter_output" "2026-05-19 21:29:05" 30.64218 1 46161895200.0926
"control_output_A" "2026-05-19 21:29:05" 106.3285 1 46161895200.0926
"Filter_output" "2026-05-19 21:29:06" 30.70061 1 46161895211.7361
"control_output_A" "2026-05-19 21:29:06" 106.4566 1 46161895211.7361
"Filter_output" "2026-05-19 21:29:07" 30.76078 1 46161895223.3565
"control_output_A" "2026-05-19 21:29:07" 108.3058 1 46161895223.3565
"Filter_output" "2026-05-19 21:29:08" 30.8273 1 46161895235.0116
"control_output_A" "2026-05-19 21:29:08" 94.74768 1 46161895235.0116
"Filter_output" "2026-05-19 21:29:09" 30.89222 1 46161895246.6551
"control_output_A" "2026-05-19 21:29:09" 97.67635 1 46161895246.6551
"Filter_output" "2026-05-19 21:29:10" 30.96314 1 46161895258.3102
"control_output_A" "2026-05-19 21:29:10" 99.32322 1 46161895258.3102

```

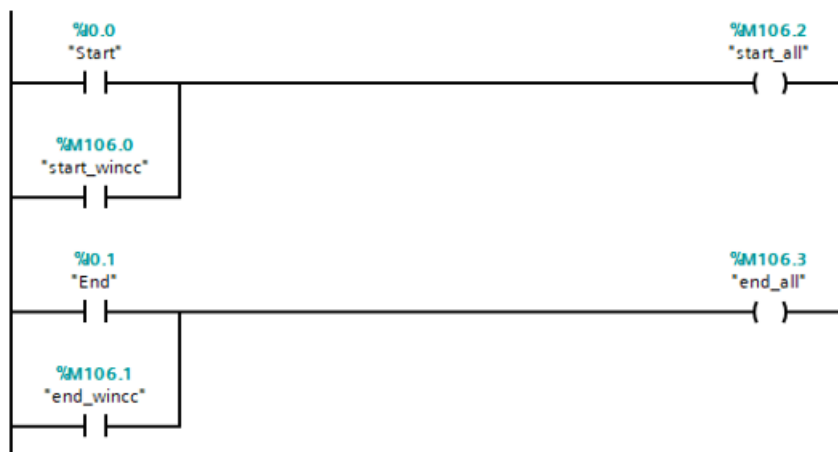
可以看到, 我们选定的数据都被记录了下来, 并且还有每条记录对应的时刻。并且以此可以将其绘制为历史记录曲线。

3.6 系统整体启动

系统所有的程序都串联上一个 Start_signal 变量, 标志着整体的启动。可搭建如下带自锁的梯形图, 通过两个非自锁的按钮可以控制系统的启动与停止。



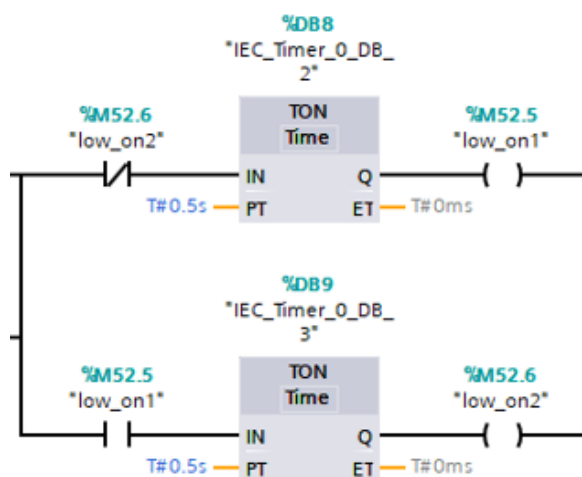
系统的整体启停不仅受物理按钮限制，也受上位机的模拟按钮控制，分别对应了两组变量。两组变量又有如下关系：



其中 Start 和 End 是外部变量，与 DI 相关联，与两个物理非自锁按钮相连。两组变量分别通过并联关系得到一组直接控制系统启停的变量 start_all 与 end_all。这样使得两组开关相互独立，无论是物理开关还是上位机开关被触发，系统都能做出反应。

3.7 温度上下限报警

上下限报警灯、蜂鸣器均通过继电器来实现隔离驱动。与其直接连接的三个外部变量依次为：TooHigh, TooLow, Buzzer。无论是指示灯的闪烁还是蜂鸣器的分频率报警，都可以通过给对应的外部变量一个相应频率的方波信号来实现。而这个方波信号可由两个 TON 实现。



如图所示，起始时 low_on2 和 low_on1 都为 0，上方 TON 先触发，0.5s 后 low_on1 置为 1，触发下方 TON，0.5s 后 low_on2 置为 1，这导致了上方的 TON 输出为 0 使 low_on1 置为 0，又导致下方 TON 输出为 0 使 low_on2 也为 0，整个系统恢复到初始状态，如此循环，low_on1 的信号变化便是一个频率为 1HZ 的方波信号。

报警的 SCL 判断代码如下，当温度超上限时触发报警，只有再低于（上限-回差）时才会停止报警；同理当温度低下限时报警，只有再高于（下限+回差）时报警停止。

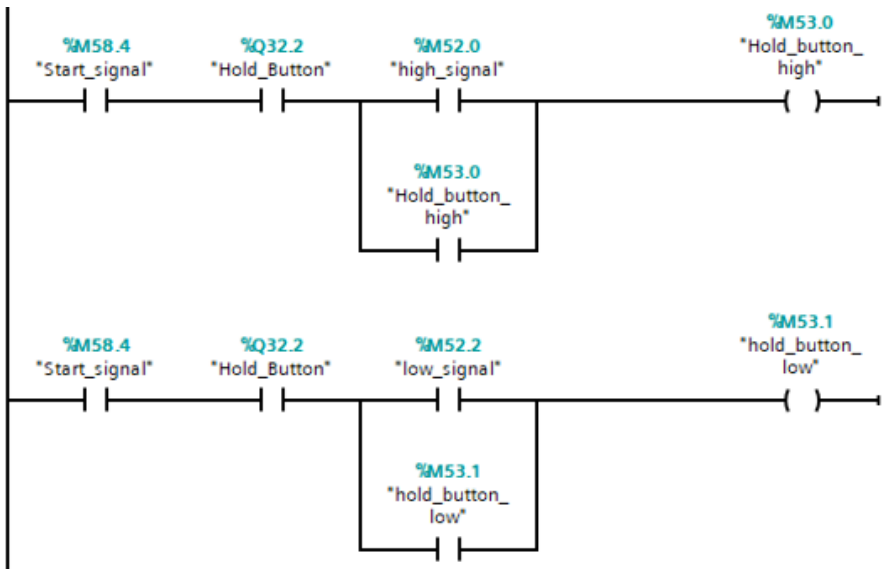
```

1 IF #Temp_Actual >= #HighLimit THEN
2     #LED_High_Out := TRUE;
3 ELSIF #Temp_Actual <= (#HighLimit - #Hysteresis) THEN
4     #LED_High_Out := FALSE;
5 END_IF;
6 IF #Temp_Actual <= #LowLimit THEN
7     #LED_Low_Out := TRUE;
8 ELSIF #Temp_Actual >= (#LowLimit + #Hysteresis) THEN
9     #LED_Low_Out := FALSE;
10 END_IF;

```

然后将对应表示报警状态的变量与对应频率的方波信号串联后再输出给外部变量即可实现声光报警。

报警状态保持：为了保证在报警时按下报警保持按钮后，即使温度在正常范围内仍然声光报警。在报警状态变量上在串联一个状态保持变量，状态保持变量又由一个自锁程序决定。同时为了区分上限报警和下限报警各自定义了一个状态保持变量。

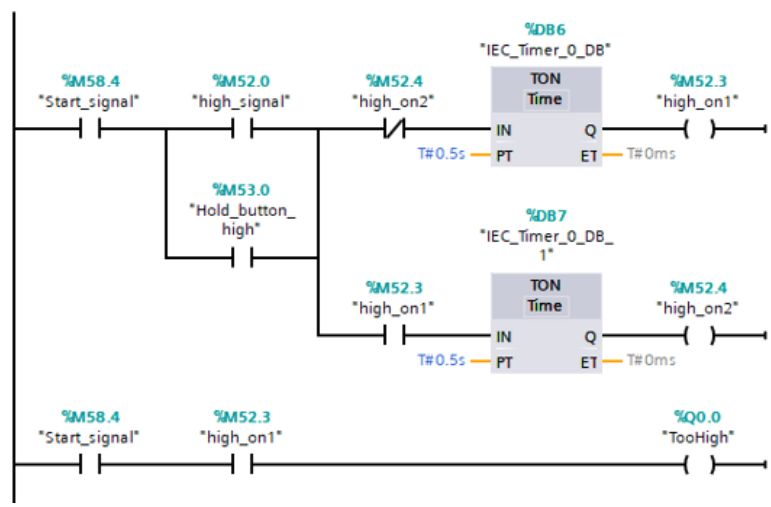


如图所示，其中 Hold_Button 变量与上位机的报警保持按钮相关，每按一下按钮此变量取反位。当其置 1 时，即使对应报警信号复位，状态保持变量依然置位。

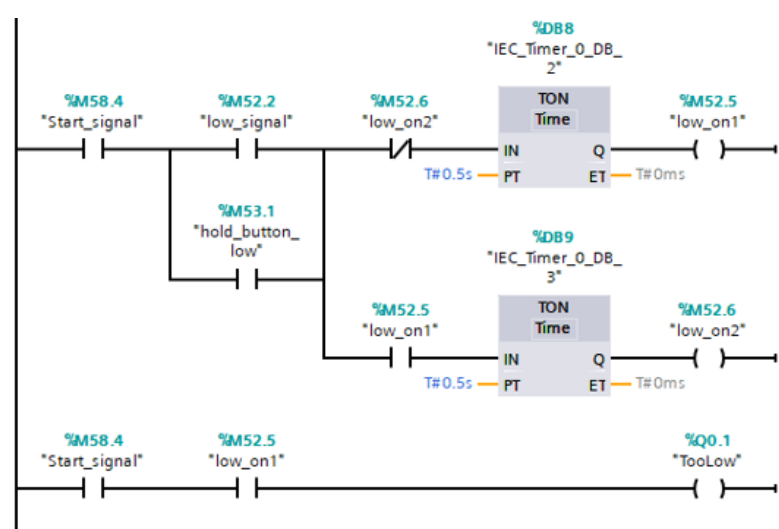
报警消音：由于报警消音只针对蜂鸣器，因此在最终将信号输出给蜂鸣器时再串联上一个 Mute_button 变量即可，此变量再关联上位机的报警消音按钮，每按一下取反位。

因此最终的报警代码如下：

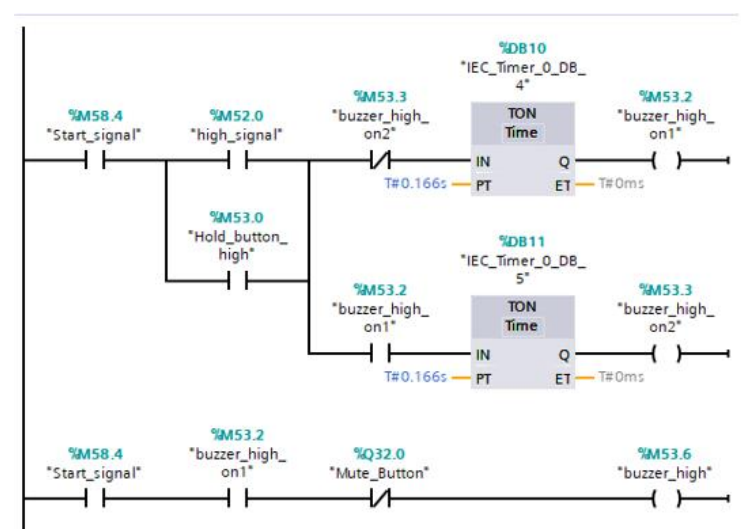
超上限的指示灯闪烁：频率 1HZ



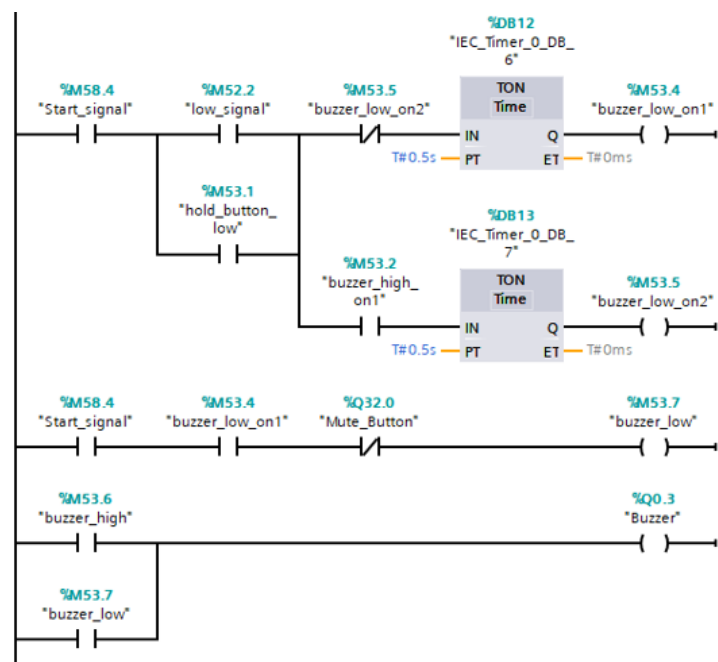
低下限的指示灯闪烁：频率 1HZ



超上限的蜂鸣器报警：频率 3HZ，TON 时间应为 1/6s



低下限的蜂鸣器报警：频率 1HZ，TON 时间应为 0.5s



3.8 传感器掉线报警与安全联锁

当传感器掉线时，系统从温度对象读取到的数据为-32768，正常情况下读取到的应该是一个范围为 0-27648 的整数。因此当测量到的温度在量程之外、突然剧烈跳变都会触发传感器故障报警。

```
// 传感器掉线报警与安全联锁
#Fault_Detected := FALSE;
#Fault_Type := 0;
//1.超量程/断线检测
IF #Temp_Actual < #Temp_min OR #Temp_Actual > #Temp_max THEN
    #Sensor_Fault := TRUE;
    #Fault_Type := 1;
    #Fault_Detected := TRUE;
END_IF;
//2.跳变异常检测
IF NOT #Fault_Detected THEN
    #Delta_Temp := ABS(#Temp_Actual - #Temp_Last1);
    IF #Delta_Temp > #Jump_Threshold THEN
        #Sensor_Fault := TRUE;
        #Fault_Type := 2;
        #Fault_Detected := TRUE;
    END_IF;
END_IF;

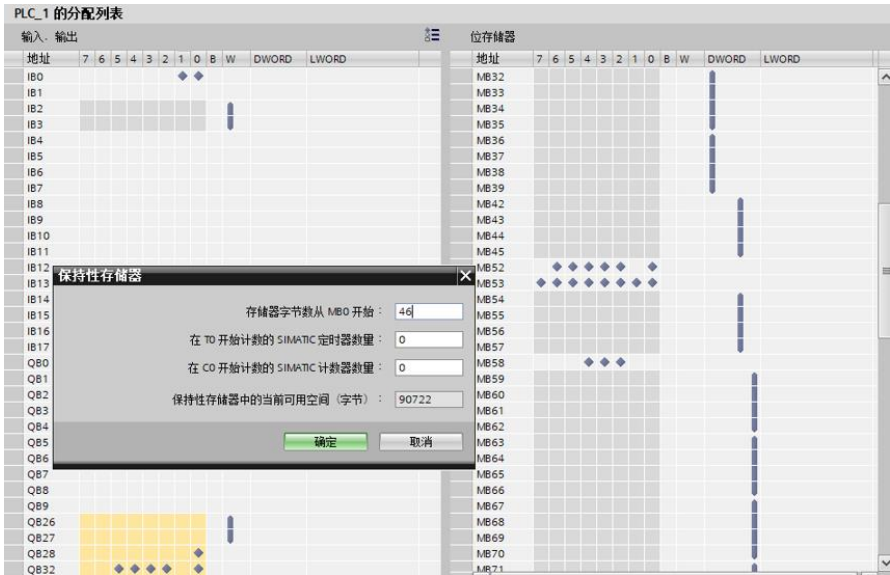
//未检测到故障则清除故障标志
IF NOT #Fault_Detected THEN
    #Sensor_Fault := FALSE;
    #Fault_Type := 0;
END_IF;
//更新历史值
#Temp_Last2 := #Temp_Last1;
#Temp_Last1 := #Temp_Actual;
#Hold_Active := #Hold_Active_Internal;
#Mute_Active := #Mute_Active_Internal;
```

安全联锁则是将系统强制切换为手动模式并输出为 0，以实现切断加热。

```
1 #auto/manual := FALSE;
2 #control_output := 0.0;
3 #confirm := FALSE;
```

3.9 参数掉电保持

掉电保持即将 cpu 断电重启时，某些变量仍然保持着断电之前的值。需要断电保持的变量有 Kp、Ti、Td、温度报警上下限、温度量程上下限、回差、设定值、手动控制值。而在 PLC 中有一块能够掉电保持的存储地址，只需将这些变量定义在这些地址中即可，此处选择的地址范围为 MB0-MB46。



四、控制系统硬件组成

4.1 I/O 统计

(1) 输入

变量名	数据类型	地址	注释
Start	Bool	%I0.0	启动开关(硬件)
End	Bool	%I0.1	停止开关(硬件)
Highs	Bool	%I0.3	温度超上限标志
Lows	Bool	%I0.4	温度低下限标志
Temperature_input	Int	%IW2	温度测量值数字量输入

(2) 输出

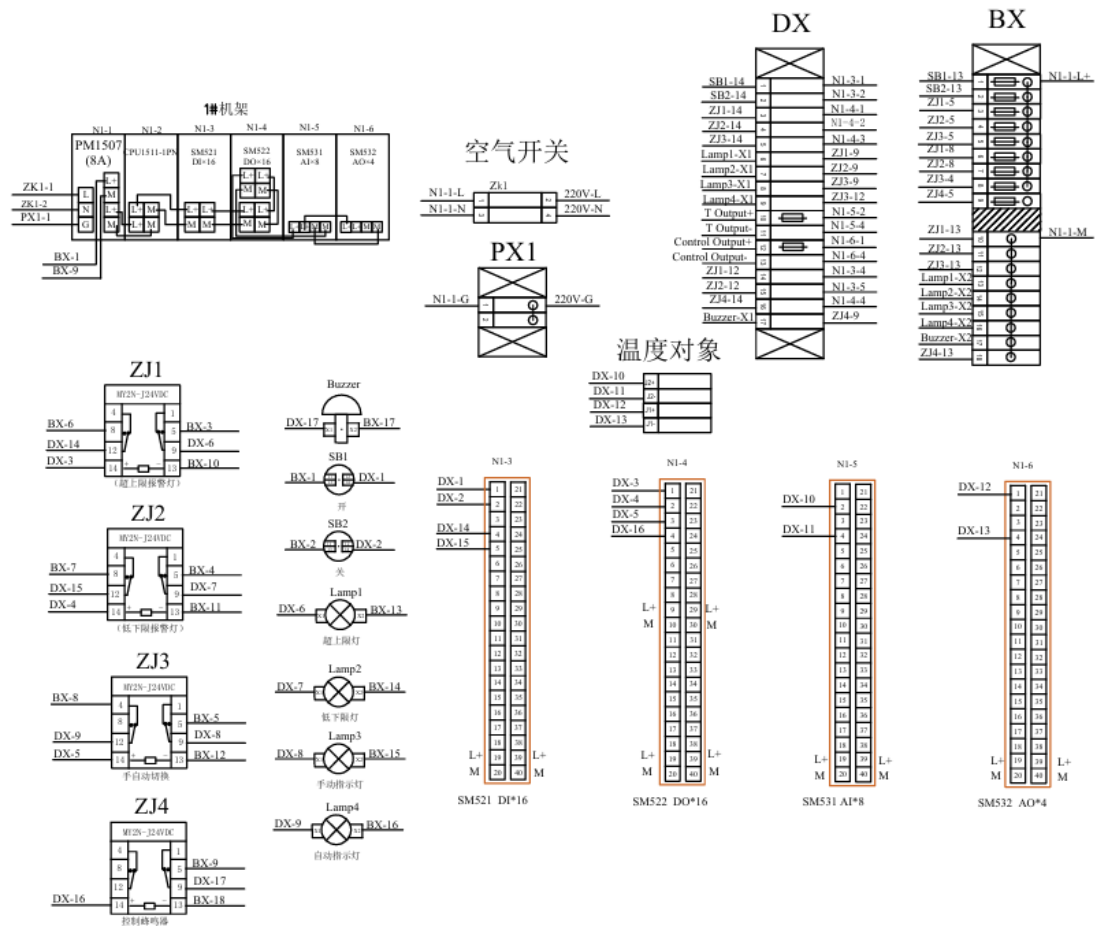
变量名	数据类型	地址	注释
TooHigh	Bool	%Q0.0	上限报警指示灯
TooLow	Bool	%Q0.1	下限报警指示灯
Auto/Manual	Bool	%Q0.2	手/自动指示灯
Buzzer	Bool	%Q0.3	蜂鸣器

变量名	数据类型	地址	注释
-----	------	----	----

Control_Output	Int	%QW2	控制器输出
----------------	-----	------	-------

4.2 硬件配置

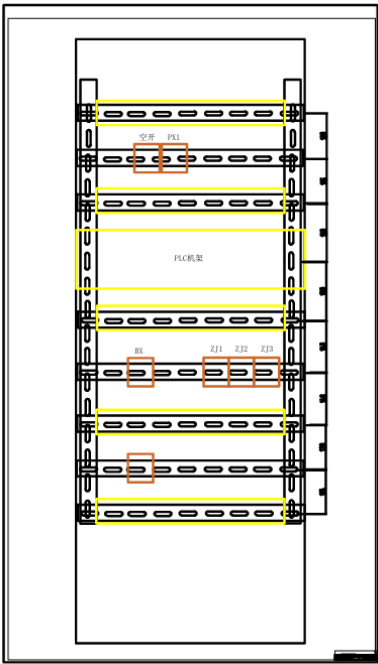
4.2.1 接线图



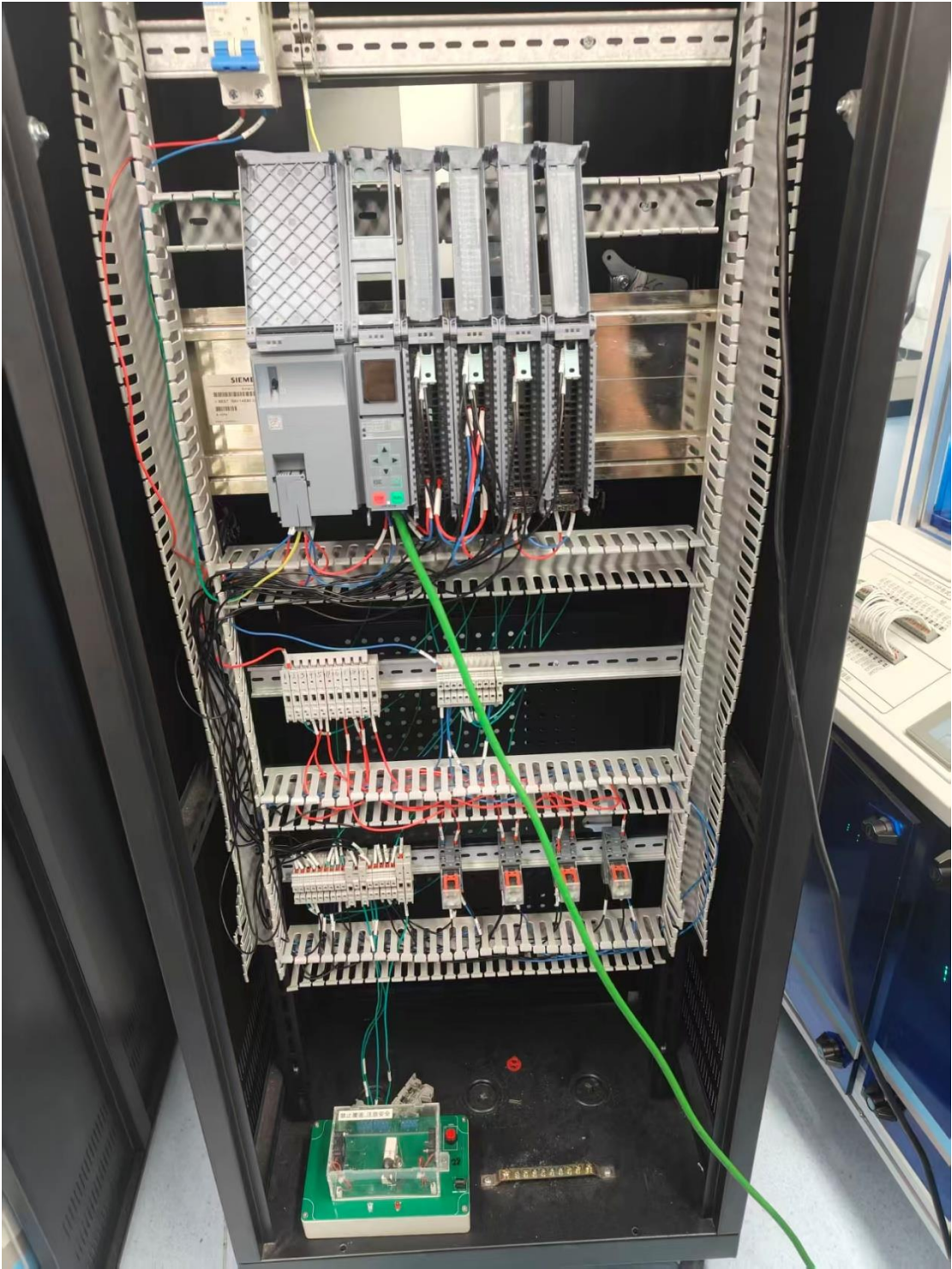
本项目采用 PM1507 电源模块,基本单元配置为 S7-1511PLC, SM521 DI-16, SM522 DO-16, SM531 AI-8, SM532 AO-4。

4.2.2 机柜背面布设

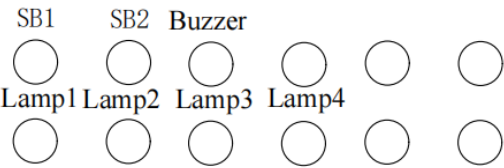
行数	硬件配置
1	空气开关、PX1
2	机架 PLC
3	接线端子 BX、继电器 ZJ1-ZJ4
4	接线端子 DX



实物接线照片：（实物接线时将继电器 ZJ1-ZJ4 移动到第四排了，与接线图略有出入）



4.2.3 机柜正面布设



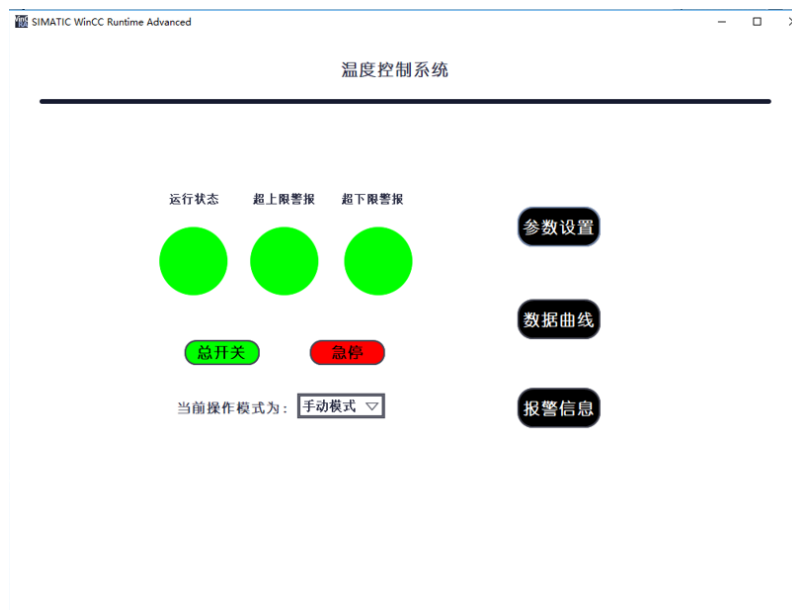
依次为启动开关、停止开关、蜂鸣器、超上限报警灯、低下限报警灯、手动状态灯、自动状态灯。



五、系统调试运行情况

5.1 主要软件调试主要界面截图

系统运行后，系统默认显示 WinCC 主界面：



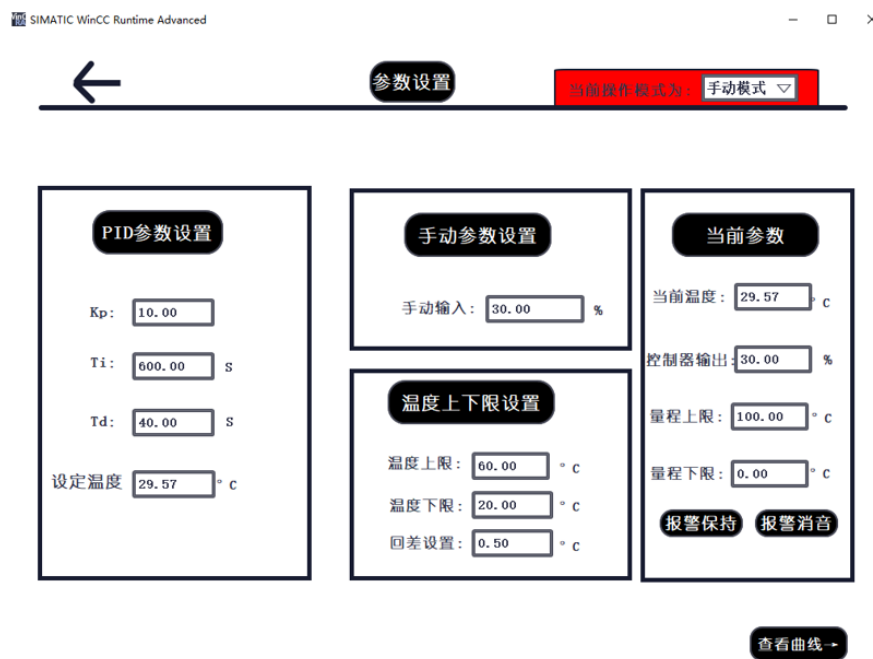
其左侧有标志运行状态的总指示灯，运行时为绿色、停止时为红色。下方有启停按钮、可控制整个系统的启停状态，都是非自锁按钮。

旁边是超上限和低下限报警指示灯，当触发报警时不仅机柜上的灯闪烁，wincc 界面报警灯也闪烁。



下方有下拉选项框可以选择自动和手动模式，右侧有三个界面跳转按钮。

参数设置界面：



左上角按钮返回到主界面，右下角按钮跳转到曲线页面。

左侧是 PID 相关参数，Kp、Ti、Td 和设定值；中间是手动控制输出、温度报警上下限与回差；右侧上方可监测当前的温度和控制器输出，下方可调整温度量程上下限，同时也有报警保持和报警消音按钮。

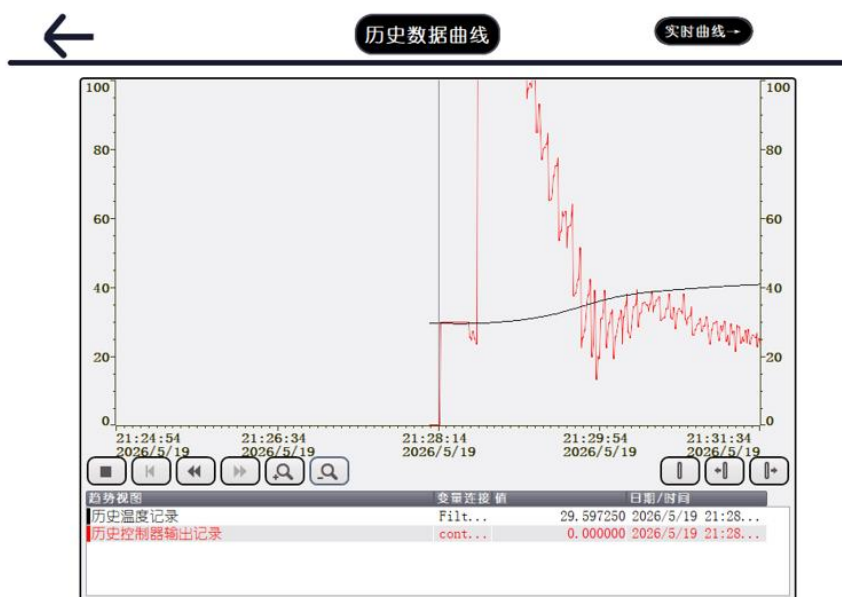
右上角按钮可切换手动和自动模式，切换后会弹出一个确认框防止误触。



曲线页面:



右上角可选择是实时数据曲线还是历史数据曲线。



报警信息界面:

The figure shows the alarm information interface in SIMATIC WinCC Runtime Advanced. It features a table of alarm records and a status indicator. The table has columns for ID, Time, Date, Status, Text, and Confirmation. The status indicator shows '当前故障状态: 无故障' (Current fault status: No fault).

编号	时间	日期	状态	文本	确认组
3	21:32:20	2026/5/19	I/O	温度超限报警!!!!	0
3	21:32:10	2026/5/19	I	温度超限报警!!!!	0
2	21:32:07	2026/5/19	I/O	温度超限报警!!!!	0
2	21:31:57	2026/5/19	I	温度超限报警!!!!	0
240000	21:30:52	2026/5/19	I	无效的许可证密钥! WinCC Logging for Runtime Advanced	0
240000	21:20:52	2026/5/19	I	无效的许可证密钥! WinCC Logging for Runtime Advanced	0
140000	21:20:40	2026/5/19	I	已建立连接: HMI_连接_3, 站 192.168.0.1, 机架 0, 插槽 1.	0
140000	21:20:40	2026/5/19	I	已建立连接: HMI_连接_1, 站 192.168.0.1, 机架 0, 插槽 1.	0
110001	21:20:37	2026/5/19	I	切换为 "在线" 操作模式.	0
70018	21:20:37	2026/5/19	I	用户管理已成功导入.	0
70022	21:20:37	2026/5/19	I	已开始导入用户管理.	0
80026	21:20:37	2026/5/19	I	记录初始化结束。所有记录正确.	0
80028	21:20:37	2026/5/19	I	记录初始化开始.	0

当温度触发报警后, 会在此留下一条报警记录, 同时右上角故障状态表示了当前传感器是否掉线故障。

5.2 调试过程中碰到的主要问题分析

问题一: 上位机的两个启停按钮绑定了外部变量 Start 和 End, 结果在上位机上按下后常常没反应, 或者偶尔才会反应一次。

分析: Start 和 End 都是 DI 的外部变量, 分别连接两个外部物理按钮。而即使物理按钮没

有被按下，这条通路仍然是存在的，只是此时按钮传过去的变量值为 FALSE，相当于系统程序每次循环时都给外部变量 Start 和 End 都赋值一次 FALSE，如果直接将上位机的按钮关联这两个外部变量，相当于变量同时有两方输入，必然会冲突。系统程序每次循环执行都是有顺序的，程序启停的代码只在每次循环的开头执行一次，如果在此次循环中这部分代码已经执行过了，此时再通过上位机给外部变量赋值 TRUE，结果下次循环一开始通过物理按钮给外部变量赋值给 FALSE 覆盖掉了先前的赋值，就会导致程序没反应。于是我们给上位机的启停开关又新定义了一组开关变量，并把这一组变量与物理开关的外部变量做并联处理，并联的结果再执行启停代码。这样两种开关方式互相独立不冲突，且均能控制整体启停。

问题二：无论如何调节 PID 变量，P 和 D 的部分总是特别小，几乎只是 I 在起作用。

分析：没有将控制算法放入固定时间中断的 OB 块，增量式 PID 算法的前提是固定时间 T_s 采样，一开始用的普通 OB 块，导致系统执行完代码立即执行下一次，导致前后间隔时间很短，相邻采样得到的误差变化极小，而和误差变化率与误差二阶变化率相关的 P 和 D 部分也极小，这就导致了 PID 控制失衡，尽管将 P 和 D 设置的非常大也不能控制。后续将所有控制、滤波相关的算法都放进了固定 0.1s 中断的 OB 块内，此时 P 和 D 能够正常发挥作用了。可得到较好的控制效果。

问题三：即使三个要求都满足了仍然不能够实现无扰动切换

分析：代码的运行是自上而下的，一开始我们的梯形图代码顺序是自上而下：自动模式时手动控制输入等于自动控制输出，手动控制代码、手动模式下设定值等于测量值、自动控制代码。而在自动控制算法中有一个首次切为自动模式将手动控制值赋值给自动控制输出的操作。但是当系统先处于手动模式时，首先手动控制代码运行、设定值追踪测量值运行，自动控制代码尚未运行、而自动控制输出值关联的变量此时为默认值 0。而系统切为自动模式的首个循环中，先执行将自动控制输出赋值给手动控制输入，但是此时自动控制输出还是默认值 0，会先将手动控制值变为 0，然后再执行自动控制中首次切自动的赋值操作，这样原本应该赋值为切换前的手动控制值变为了赋值了 0，就不能实现无扰动切换了。将其在主程序的顺序调整为：手动模式下设定值等于测量值、自动控制代码、自动模式时手动控制输入等于自动控制输出，手动控制代码即可。

六、系统优势

6.1 增量式 PID，便于手自动切换，自带抗积分饱和

增量式 PID 控制器输出的是控制增量，当进行手自动模式切换时，控制器不会突然输出一个很大的控制量，从而避免了对被控对象的强烈冲击，只需在切自动模式的首次循环时将手动控制值赋值给自动控制的基准值便可实现无扰动切换。此外增量式 PID 不存在积分求和项、不会不断累积长时间的误差、因此不会出现积分饱和的情况。

6.2 系统参数调整方便

在上位机界面不仅可以在线调整 PID、设定值、手动控制值等参数，也可以远程启停程序、观察程序运行状态，自动存储运行记录与报警记录。

6.3 鲁棒性好

滤波算法对量程的适应、手自动切换的二级确认弹窗、将最终输出给对象的控制值单独定义一个函数来选择、基于梯形图和 TON 实现的方波信号、掉线安全联锁、参数掉电保持等都提升了代码的鲁棒性，使其可以应对各种情况稳定运行。

七、验收与改进

7.1 验收的故障检查

验收时我们发现虽然在监控表在 Control_output 能够正确得到数值，但是温度对象没有做出反应，检查发现 Control_output 的地址从%QW2 变为了%QW4，与我们的实物连线不符。另外发现温度对象控制是否加热的跳线帽位置也改变了，此外检查控制代码时发现原本将输出限制在 0-100 的限幅代码也消失了，全部修正后可以正常运行了。

7.2 改进功能

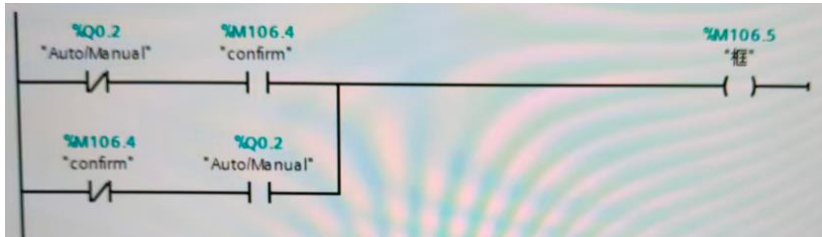
在验收时，老师也针对我们的项目提出了许多改进建议：

- 1) 滤波算法不要默认只映射到 0-100℃，而是要根据量程改变，如果量程改为-20-80℃，虽然得到的测量量还是 0-27648，但是其代表的物理意义范围已经改变了。

```
#Temp_ana:=NORM_X(MIN:=0,VALUE:=#Temp_in,MAX:=27648);  
#Ana_In := SCALE_X(MIN := #temp_min, VALUE := #Temp_ana, MAX := #temp_max);
```

于是我们将第二次映射的 MIN 值和 MAX 值从 0 和 100 改变为了量程的下限与上限变量。

- 2) 手动自动模式切换时要加一个确认弹窗，在实际工况中，这里要做防误触设计。



新定义两个变量 confirm 和框，框与上位机界面确认框的出现相关联，初始状态两者都为 FALSE，此时为手动，切换为自动时 Auto/Manual 为 TRUE，此时下方通路开启出现框，框上有两个选项取消和确认，按下取消后令 Auto/Manual 复位为 0 框消失，按下确认后 confirm 为 TRUE，上下都断路框消失，以此类推，每次按下确认键 confirm 都取反位。

- 3) 要在上位机上也添加总启停按钮，因为在实际工况中有时上位机离现场物理按钮较远，为便于操作同时也为了出现紧急情况时能够快速做出反应，应该在上位机上也添加启停按钮。具体实现如 5.2.1 所示。

4)

八、心得体会

李丰克：我本次计控实践中负责接线图、手自动控制算法以及部分报警算法。在绘制接线图的过程中，我对西门子 PLC 供电接法、继电器的电气隔离应用以及 DI、DO、AI、AO 各通道的四线制接法有了更深刻的理解，尤其是 AI 和 AO 的通道电路原理的区别导致接线方式不同。而对于滤波、手自动控制算法，需要注意的一点是转化为直接输出给对象的变量时需要模数转换，自动控制算法因为一开始没有将算法写在固定时间中断的 OB 块中导致增量式 PID 的 PD 部分效果特别差，被迫改为位置式，后改回了更简单且无积分饱和的增量式。在写算法时最令我印象深刻的是无扰动切换，理论上只需要满足三条要求便可以实现，但是在实际实践时却会遇到各种问题，甚至连梯形图的顺序都会影响无扰动切换的效果。最后的报警阶段，对于需要闪烁的报警灯和有不同 HZ 输出的蜂鸣器，需要给元件输出一个对应频率的方波信号，对于这个方波信号，我们一开始想用系统自带的一系列 clock 变量，但是后续发现了仅需要梯形图用两个 TON 便可实现方波信号，并且发现报警保持和报警消音均可用梯形图巧妙实现，于是我们选择用梯形图更优雅的实现了报警系统。本次实验中我主要负责了较为核心的算法内容，让我深刻体会到理论与实践的区别，在具体工况中会遇到各种未曾设想到的问题，本次计控实践不仅加深了我对课程内容的理解，也使我认识到了实际工况现场需要考虑各种实际问题，